„Workshop Portal” – fejlesztési feladat

(AngularJS 1.x + TypeScript + Bootstrap 5)

Szabadon választott alkalmazás – követelményrendszer (AngularJS 1.x + TypeScript)

Kötelező funkcionális elemek

Adatlista megjelenítése

- Legyen egy lista (pl. termékek, feladatok, résztvevők, jegyek stb.), amelynek **tartalmát TypeScript kód** állítja elő vagy tölti be (statikus tömbből vagy lokális JSON-ból). A lista AngularJS-sel jelenjen meg.

Elemek hozzáadása / módosítása legalább egy ponton

- Az alkalmazás biztosítson olyan műveletet, amellyel a felhasználó új elemet vehet fel vagy egy meglévőt módosíthat (pl. űrlap beküldéssel). Üres vagy hibás adat **ne** kerülhessen a listába.

Eseményfigyelés

- Legalább **két különböző** felhasználói eseményre reagáljon (pl. gombkattintás, Enter lenyomása űrlapmezőben, választás egy legördülőből). Az események hatása legyen látható (lista frissül, számítás lefut, üzenet megjelenik).

Kétirányú adatkötés és interpoláció

- A felületen legalább két helyen mutassa meg a modell aktuális értékét (pl. előnézet, számláló, összegzés).

Szűrés vagy keresés

- Valós idejű szűrőmező (kis-/nagybetűfüggetlen), amely a listát leszűkíti.

Számított érték(ek)

- Legyen legalább egy **számítás** (összegzés, árkalkuláció, pontszám, státusz-logika stb.). A számítást **TypeScript modul** végezze (pl. computeTotal, score, toInitials), és a sablon csak az eredményt jelenítse meg.

Űrlap-validáció és visszajelzés

- Legalább egy űrlapmezőn legyen formai ellenőrzés (pl. kötelező, e-mail minta, minimum hossz). Az érvénytelen állapot látszódjon; a beküldés gomb legyen letiltva hibás állapotban.

Vizuális visszajelzés adatokból

- Dinamikus stílus vagy osztály (pl. „kész” elem áthúzva; státusz színe; előnézeti négyzet színe kiválasztás szerint).

Bootstrap 5 használata

- Rendezett elrendezés (container, row/col), űrlapok, gombok, kártyák.

Kötelező technikai elemek

- **AngularJS 1.x** használata (ControllerAs minta javasolt), sablonokban csak **olcsó** logika.
- **TypeScript modul(ok)** a számításokra/ellenőrzésekre; a fordított JS egy bundliban vagy külön fájlokban legyen betöltve a HTML előtt.
- **Projekt-szerkezet** áttekinthető mappákkal (nézetek, vezérlők/komponensek, TS-források, fordított JS).
- **Dokumentáció (README)** rövid futtatási útmutatóval: hogyan nyitja meg, hogyan fordít TS-t, mit hol talál.

Választható (ajánlott) kiegészítések – Nem kötelező!

- **ng-options** használata objektum-alapú <select>-hez (modell = teljes objektum).
- **Állapot megőrzés** böngészőben (localStorage) – nem kötelező.
- **Kliensoldali útvonalkezelés** (ngRoute) 2–3 aloldalra bontva.
- **Statikus JSON betöltése** (pl. assets/data.json) és TS-en keresztüli előfeldolgozása.

Tiltások / kerülendők

- Szerveroldali komponens, adatbázis, bejelentkezés (nem cél).
- Minden logika sablonba zsúfolása; a számítások TypeScriptben legyenek.
- Ne használjon külső állapotkezelőt vagy más keretrendszert (maradjon AngularJS + TS).

Leadandók

- Teljes forráskód áttekinthető szerkezettel.
- Fordított TypeScript fájl(ok) (pl. dist/app-bundle.js).
- Rövid **README**: futtatás (pl. VS Code Live Server), TS-fordítás, fő funkciók.

Értékelési szempontok

- **Funkcionális megfelelés**: minden kötelező elem megvan és működik; a lista TS-ből jön; események érzékelhetők; számítások helyesek.
- **Kódminőség**: elkülönített felelősségek, tiszta ControllerAs minta, érthető elnevezések.

- **Használhatóság/kinézet**: Bootstrap, következetes UI; visszajelzések; letiltott gombok, ha indokolt.
- **Dokumentáció**: futtatási lépések, rövid ismertetés.

Példa elfogadási próbák (bármely témára alkalmazhatók)

- Lista üres állapotban kulturált üzenetet mutat; új elem felvétele után azonnal megjelenik.
- Üres vagy hibás bevitel **nem** kerül a listába; a mentés gomb ilyenkor le van tiltva.
- Keresésre a lista valós időben szűkül; kis-/nagybetű nem számít.
- Legalább két eltérő esemény (pl. gomb + Enter) ugyanazt a műveletet képes kiváltani.
- Számított mező(ke)t a TS adja vissza; szélső értékek (üres, negatív, „abc”) stabilan kezelve.
- Dinamikus stílus/osztály a modell állapotából következik (pl. kész elem áthúzott).

Ön dönti el a témát (jegykezelő, mini bolt, receptgyűjtő, eseményregiszter, olvasmánylista stb.). A lényeg, hogy a fenti funkcionális és technikai feltételek teljesüljenek, a számítások/ellenőrzések pedig **TypeScriptben** legyenek megírva, és a felület AngularJS-szel, Bootstrap-tel kulturáltan jelenjen meg.